Akshansh Agrawal
SRN: PES2UG24CS045
Lab 3: Component Modelling & Architectural Pattern Selection
System: OpenAPI Mock Server Generator

Written Justification

Architecture Selection: We chose a Layered Architecture for the OpenAPI Mock Server Generator system.

Architectural Choice

The system is built as three layers: a Presentation layer (Dashboard UI), a Business layer (Spec Manager, Mock Engine, and Response Validator), and a Data layer (Data Store). A request moves down through the layers and a response moves back up. The UI never talks to the database directly. It goes through the business components, which is the standard rule of a layered design.

Two Reasons for This Choice

1. It matches the natural request lifecycle. The tool does its work in clear stages. First it parses and validates the spec, then it generates the mock endpoints, then it builds a response, and finally it validates that response against the schema. Each stage maps cleanly to one component in the business layer, so the code stays organised around what the system actually does.

2. Each layer can be tested and changed on its own. Because the layers are separated, we can test the Spec Manager or the Response Validator without running the UI. We can also swap the Data Store from a file to a real database later without touching the Mock Engine, since they only talk through a fixed interface. This is useful for a developer tool that people will extend over time.

Security Advantage

The Data layer sits behind the Business layer, so the Dashboard UI cannot reach the stored specs, endpoint configuration, or request logs on its own. Every read and write has to pass through the business components, which gives us one place to check access and validate input. Uploaded specs are also validated by the Spec Manager before anything is stored, so a broken or malicious file is rejected early instead of being saved and served.

Performance Benefit

The Mock Engine is its own layer, so we can tune and cache it without affecting the rest of the system. Parsed specs and generated endpoints can be held in memory in that layer, which lets the engine answer requests quickly instead of reparsing the spec each time. This helps the system reach its target of 1,000 requests per second (NFR-001), since the hot path stays inside one focused component.